Doc. no: DTU-30082-002
Rev: 1.2
Date: 2019-01-16

Projektarbejde i Digitaldesign
LabviewOscilloscope Manual

Udarbejdet af:

Peter Brauer 2019-01-16

Template no: 2

Contents

1 Introduktion 3

2 Labview 4
2.1 Installation 4
2.2 Oskilloskopprogrammet 4
2.3 Opstart 4

3 Frontpanel 5
3.1 Setup Fane 5
3.2 Generator Tab 6
3.3 Oscilloscope Tab 7
3.4 Bodeplot 8
3.5 Raw Data Tab 10
3.6 Data Protocol 10

4 Blokdiagram 12
4.1 LabVIEW basics 12
4.2 Telecommand Løkken 14
4.3 Telemetry Løkken 15

5 Checksum 16
5.1 ZERO16 16
5.2 LRC8 16
5.3 CRC16-CCITT 17

6 Simulator 18
6.1 SimulatorOscilloscope 18
6.2 SimulatorControlOscilloscope 18

1 Introduktion

Til heldagsperioden I kursus 30082 skal der laves et projekt hvor der indgår en implementering af et oscilloskop. Selve oscilloskophardwaren skal implementeres på et microcontroller øvelseskort mens visningen af oscilloskopbilledet skal ske på en computer. Microcontrolleren kommunikerer med computeren gennem et serielkabel. For at få informationen ført rigtig over skal man overholde en fastlagt protokol som er vedhæftet som appendiks til dette dokument. Programmet kaldes LabviewOscilloscope og kan findes Campusnet under kursus 30082.

2 Labview

Labview er et design værktøj til at lave software applikationer på computere. Det er baseret på et grafisk programmeringssprog der kandes G og bruges ofte til at lave dataopsamlings systemer til alle former for måletekniske opgaver. Fordelen ved G er at det er intuitivt og nemt og hurtigt at lave mindre systemer men samtidig udføre forholdsvis komplekser opgaver. Det grafiske spprog er baseret på de programmerings strukturer (IF sætninger, FOR løkker, Integer variable….) som man kender fra traditionelle programmeringssprog som C og Java. I G sammensættes programmer som et grafisk ”Block Diagram” som har forbindelse til en ”Frontpage”. Programmet udføres via Frontpage som sætter det underlæggende blokdiagram i gang i henhold til de input man laver på frontpanelet.

2.1 Installation

Labview er installeret på Databarcomputerne og kan bruges herfra under hele projektet. Som DTU studerende har man adgang til at installerer programmet på sin egen computer. Installationsfilerne kan hentes på g-databar (http://www.gbar.dtu.dk/faq/21-labview) hvor det også fremgår hvordan programmet installeres. Programmet fylder ca. 3 Gb når det er installeret på computeren. Det kan være nødvendigt at installere en ekstra driver end det der følger med i standard installationen. Driveren (NIVISA1501) er til seriel porten som benyttes til kommunikation med microcontrolleren i projektet og kan findes på CampusNet under kurset.

2.2 Oskilloskopprogrammet

Programmet der skal bruges under projektet kaldes ”LabviewOscilloscope” og kan findes på CampusNet under kursus 30082. Programmet skal hentes ned i en passende folder på den computer hvorfra programmet skal afvikles.

2.3 Opstart

Labview mijøet startes op ved at doublet clikke på ”NI LabVIEW 2014”. Programmet er normalt placeret under ”National Instruments”. LabVIEW starter op med velkomst billedet vist i *Figure 1*. Programmet ”LabviewOscilloscope” hentes ind i LabVIEW ved hjælp af ”Open Excisting”

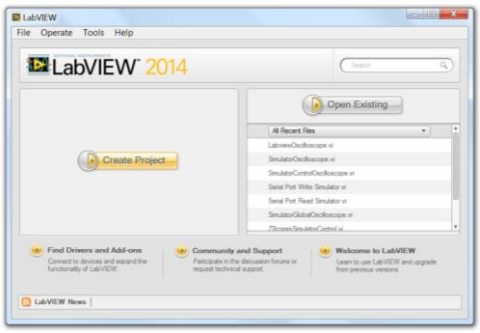


Figure 1: Opstarts billed af LabVIEW miljøet

3 Frontpanel

Når programmet er hentet ind i LabVIEW kommer frontpanelet af programmet frem som vist på *Figure 2* a.

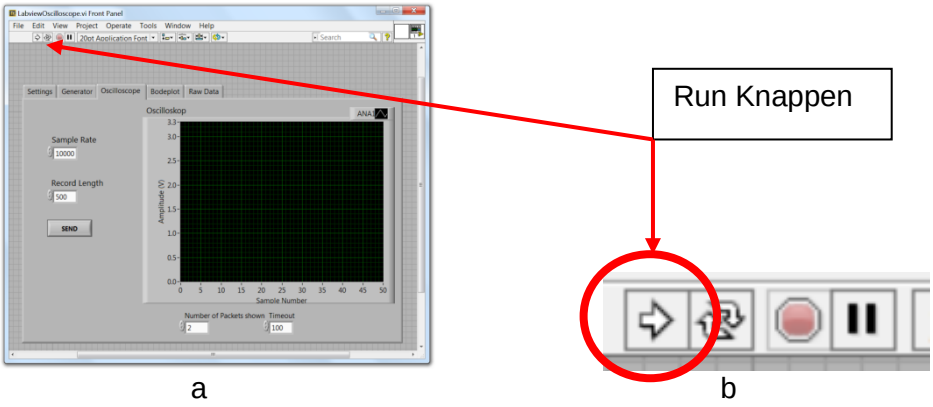


Figure 2 a: Frontpanel af LabviewOscilloscope b: "Run" knappen

Programmet startes ved at trykke på "Run" knappen (*Figure 2* b)
Frontpanelet af programmet består af fem faner som beskrevet nedenfor. Det starter op i Oscilloskop fanen og man kan skifte mellem de forskellige faner ved at trykke på dem.

3.1 Setup Fane

Under "Settings" fanen er der mulighed for at konfigurerer LabviewOscilloscope programmet (se *Figure 3*). Man kan blandt andet vælge hvilken seriel port (COM1, COM2…) der skal bruges. Man kan også vælge baud raten for denne port. Timeout er en parameter der angiver hvor længe seriel porten skal forsøge at hente data før den giver op. Det er rart at have denne parameter lav (100ms) under debug af kommunikationen. Senere bør den sættes op (10000 ms) når man skal undersøge synkroniserings fejl.

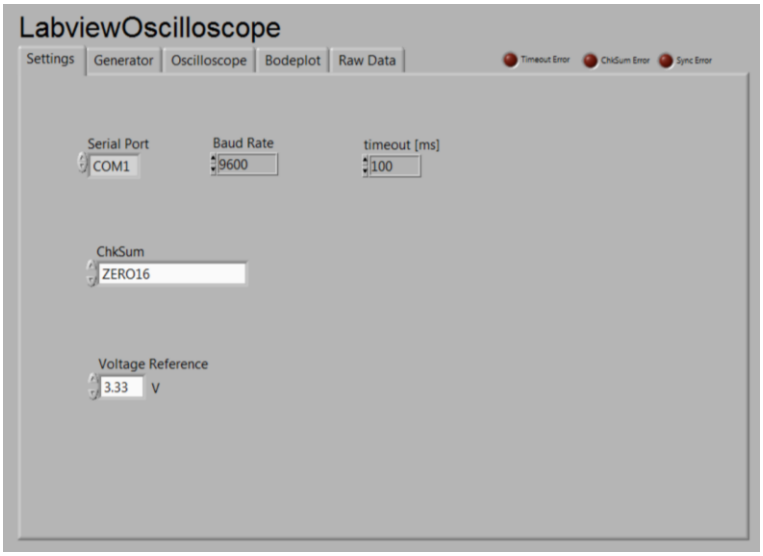


Figure 3 Under Settings-fanen kan man bl.a. opsætte seriel proten

Under ChkSum kan man vælge hvilken type ”check sum” algoritme man ønsker at bruge til kommunikationen. Det anbefales at starte med ”ZERO16”. Her skal de to checksum bytes altid være nul (0x0000). Algoritmen ”LRC8” er en simpel sekventiel bitvis xor af alle bytes i datapakken. Algoritmen ”CRC16-CCITT” er en mere avancerede ”Cyclic Redundancy Check” og er mere effektiv til at finde eventuelle fejl i datapakken. Det kræver at man sætter sig ind i CRC algoritmerne hvis man vil bruge denne funktion.
Parameteren ”Voltage Reference” bruges til at justere den maksimale spændning som kan måles med microcontrollerens A/D konverter. En digitale værdi på 255 (11111111_{Bin}) svare til denne spænding. Det forudsættes at en digital værdi på 0 (00000000_{Bin}) svare til 0 V.

3.2 Generator Tab

Under “Generator” fanen er det meningen at kan man skal kunne indstille signalgeneratoren som implementeres på FPGA kortet.

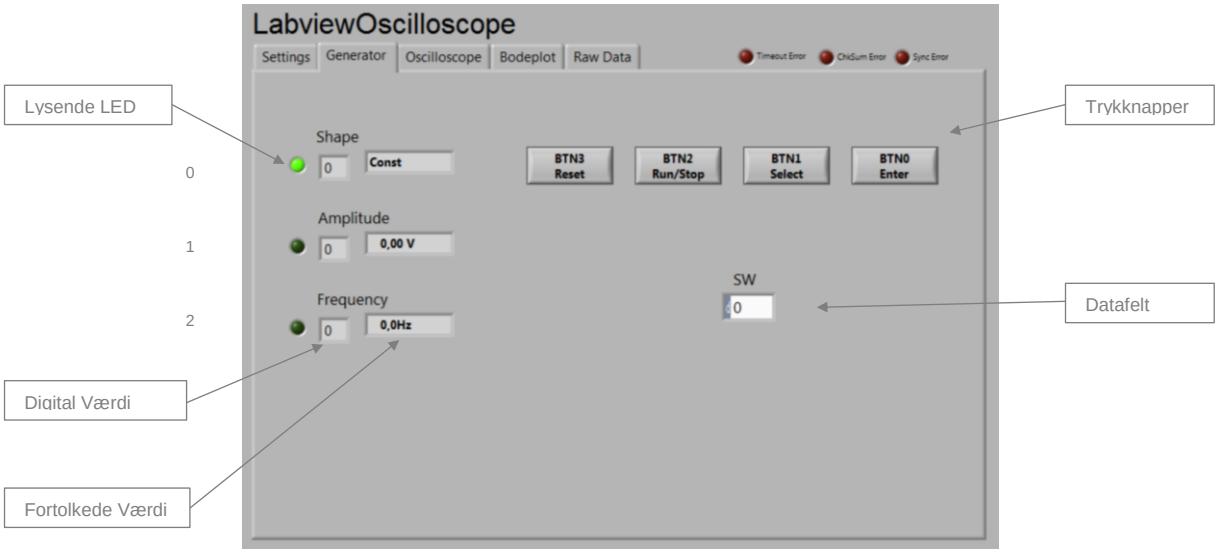


Figure 4 Generator-fanen kan man opsætte signalgeneratoren

Fanen indeholder fire trykknapper der er betegnet BTN0, BTN1, BTN2 og BTN3. Ved tryk på en af disse knapper sender LabviewOscilloscope programmet en datapakke til microcontrolleren med information om hvilken knap der blev trykket (se afsnit 3.6). Det er meningen at microcontrolleren skal fortolke trykket og sende videre til signalgeneratoren (som er implementeret på FPGA kortet) hvilken frekvens, amplitude og kurveform som signalgeneratoren skal generere. SW feltet er et numerisk datafelt hvor man f.eks. kan vælge amplituden af signalgeneratorens signal (i form af en 8-bit digital værdi ~ 0-255). Ved tryk på BTN0 (Enter) bliver denne information ført over til microcontrolleren som så skal overføre det til FPGA’en gennem SPI-forbindelsen. Ved tryk på BTN1 kan man skifte mellem om det er kurveformen (Shape), amplituden (Amplitude) eller frekvensen (Frequency) som kan indstilles ved hjælp af BTN0.

I venstre side er der en oversigt over de tre parametre som kan indstilles i signalgeneratoren implementeret på FPGA-kortet. Her kan brugeren følge med i hvilke værdier af Shape, Amplitude og Frequency der er sendt til FPGA-kortet. Der er både vist den digitale værdi af de tre parametre (0-255) samt den fysiske værdi. Den fysiske værdi bliver opdateret automatisk ud fra en given

digital værdi og svare til den effekt som den digitale værdi har i PWM implementeringen på FPGA kortet. *Table 1* viser en oversigt over parametrene i signalgeneratoren som den er implementeret på FPGA kortet.

Table 1: Oversigt over parameter til signalgeneratoren

Parameter	Digital Værdi	Fysisk værdi
Shape	0 - 3	0 ~ Konstant signal 1 ~ Firkant signal 2 ~ Savtak signal 3 ~ Sinus signal
Amplitude	0 - 255	0 – 3.3V
Frequency	0 - 255	0 – 22 kHz

Frekvensen ændrer sig ikke lineært med den digitale værdi. Den følger nedenstående formel:

$F_{Digital} = [f_7\ f_6\ f_5\ f_4\ f_3\ f_2\ f_1\ f_0]$ binary

$F_{Frequency} = [0\ 0\ f_7\ f_6\ f_5\ f_4\ 0\ f_3\ f_2\ 0\ f_1\ f_0] * 24$ [Hz]

Ovenstående transformation giver en tilnærmet logaritmisk sammenhæng mellem $F_{Frequency}$ og $F_{Digital}$ og er kodet ind i både LabviewOcilloscope og i VHDL koden for signalgeneratoren som er udleveret til projektet.

Denne del af frontpanel (venstre side) bliver kun opdateret når der sendes telekommandoer fra microcontrolleren til LabviewOcilloscope (se afsnit 3.6). Udover de tre digitale værdier sendes der også information om hvor den ”lysene” LED befinder sig.

Det er således meningen at trykknapinformationen sendes til microcontrolleren som så fortolker om den lysende LED skal flytte sig eller om informationen fra datafeltet skal overføres til den parameter hvor den lysende LED befinder sig. Dette sendes så tilbage til LabviewOcilloscope der opdaterer venstre side af generator fanen.

3.3 Oscilloscope Tab

Under “Oscilloscope” fanen er det meningen at kan man skal kunne se data der er opsamlet fra microcontrollerens A/D konverter.

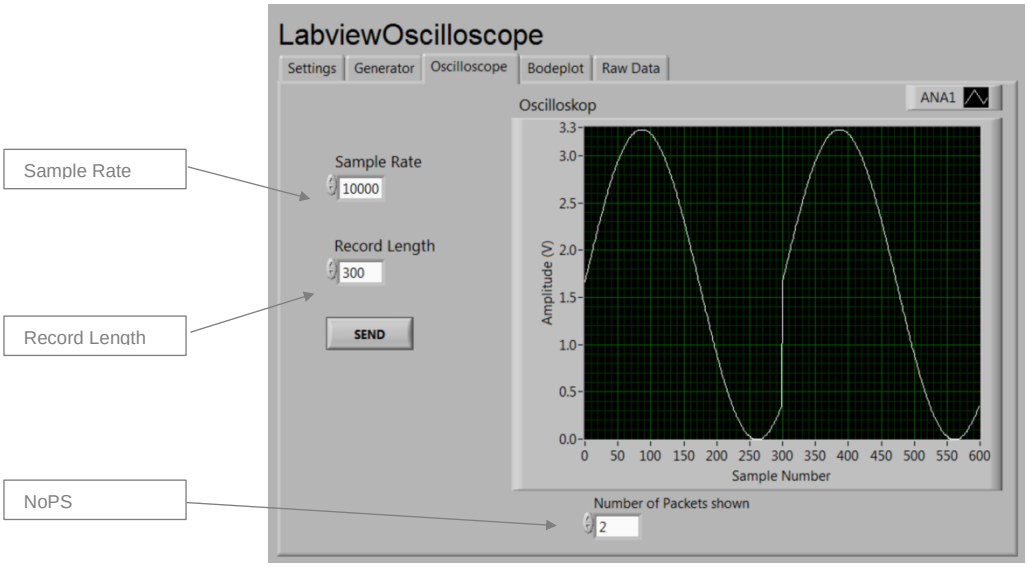


Figure 5 Oscilloscope-fanen viser oscilloskopdata

De opsamlede data sendes fra mikrocontrolleren i en pakkestruktur som angivet i Annex A. Målinger fra A/D konverteren antages at være én byte lang. Værdier fra 0-255 svare til spændinger fra 0 - 3.3V. Målingerne sendes normalt ikke som enkelt måliner men sendes i ”klumper” af 10-1000 bytes ad gangen i en enkelt datapakke. Antallet af bytes der sendes ad gangen kaldes her ”Record Length”. I ovenstående eksempel (*Figure 5*) er record length 300. På x-aksen i *Figure 5* ses sample nummer (A/D konverter målinger). De første 300 målinger er fra den seneste pakke mens de efterfølgende 300 målinger (sample nummer 301-600) er fra pakken før. Der er således vist data fra de seneste to pakker. Man kan ændre hvor mange pakker man vil se ad gangen ved at ændre NoPS i *Figure 5*. I *Figure 5* ses måling fra et sinusformet signal fra signalgeneratoren. Ved overgangen mellem de to pakker ses dataene ikke at hænge sammen. Det skyldes at der går tid mellem dataopsamlingen i de to pakker. De viste date er således ikke kontinuert. En stor del af projektet går ud på at kunne vise kontinuerte data (uden hak mellem to datapakker som vist på *Figure 5*).

Feltet ”Record Length” på *Figure 5* skal bruges til at bede microcontroleren om at ændre antallet af samples (A/D konverterings målinger) i en datapakke. Hvis den ændres fra 300 til 100 vil microcontrolleren sende datapakker 3 gange hurtigere men kun med 1/3 antal samples – altså samme antal samples per sekund. Netop antallet af samples per sekund kan ændres i feltet ”Sample Rate”. Når der trykkes på ”SEND” knappen sendes en datapakke til microcontrolleren der så skal ændre de to parametre ”Record Length” og ”Sample Rate”. Formatet på datapakken fremgår af Annex A.

Fra *Figure 5* kan man se at en hel periode af sinusformen svare til ca. 333 samples. Med en sample rate på 10.000 sps (samples pr sekund) giver det en periode tid på 33.3 ms hvilket svare til en frekvens på 30 Hz. Hvis timingen i microcontrolleren er korrekt har det analoge sinussignal fra signalgeneratoren således i henhold til *Figure 5* en frekvens på 30 Hz.

3.4 Bodeplot

Under ”Bodeplot” fanen er det meningen at man skal kunne få lavet et plot af signal amplituden som funktion af frekvensen

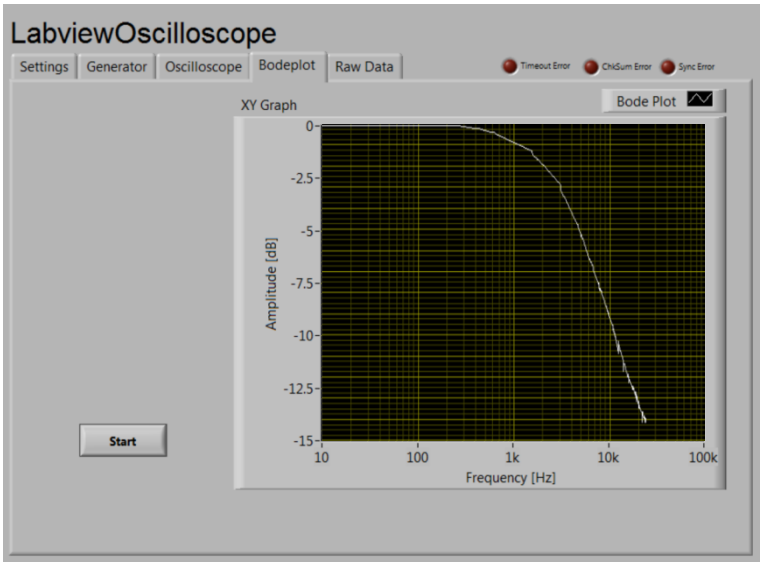


Figure 6 Bodeplot-fanen viser amplitude som funktion af frekvens

Et ”bode plot” viser amplituden af et sinussignal som funktion af frekvensen. Princippet bag målingen er skitseret i *Figure 7*. Ideen er at det analoge signal fra signal generatoren (efter PWM filteret) sendes igennem et filter som man ønsker at undersøge.

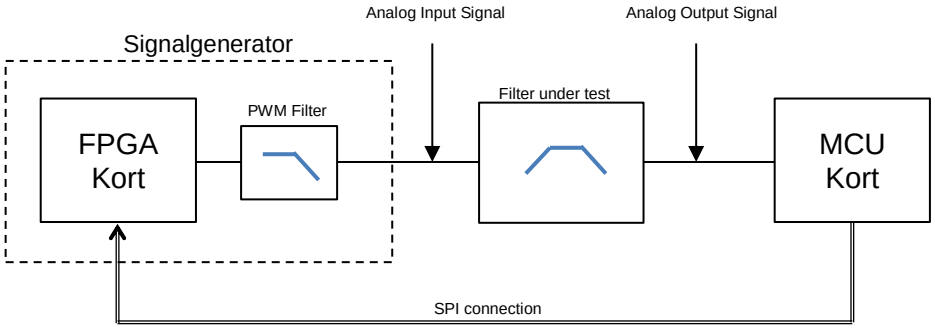


Figure 7 Princippet bag bode-plot funktionen

I *Figure 7* ses testfilteret med et analog sinussignal der går gennem filteret. Signalet bliver dæmpet forskelligt afhængig af sinussignalets frekvens. Dæmpningen (som funktion af frekvensen) beskriver filterets overføringsfunktion og det er det som bode-plottet udtrykker.

Målingen i *Figure 6* er et bode-plot fra et 1kHz lav-pas test-filter (PWM filteret er et 40kHz lav-pas filter af 4 orden og vil ikke have indflydelse på bode-plottet). Der sendes altid 255 målinger ad gangen (se Annex A). Microcontrolleren beder signalgeneratoren om at lave et sinussignal ved den laveste frekvens ($F_{Digital}$ = 1 ~ $F_{Frekvens}$ ≈ 24Hz) og med maksimal amplitude. Efter sinussignalet har passeret lav-pas filteret, måles amplituden af dette signal med microcontrolleren (dette bruges som reference). Frekvensen tælles nu én op i signalgeneratoren og amplituden måles igen af microcontrolleren. Dette repeteres for alle 255 frekvenser og amplituden omregnes til en relativ værdi i forhold til reference amplituden. Disse værdier sendes til LabviewOscilloscope gennem en datapakke og plottet i *Figure 6* opdateres.

Når der trykkes på ”Start” sendes en datapakke til microcontrolleren om at starte bode-plot funktionen og når microcontrolleren har scannet alle frekvenser sender den en bode-plot pakke tilbage til LabviewOcilloscope (se afsnit 3.6).

3.5 Raw Data Tab

Under fanen ”Raw Data” kan man se de rå telekomandodata der bliver sendt fra microcontrolleren til LabviewOscilloscope.

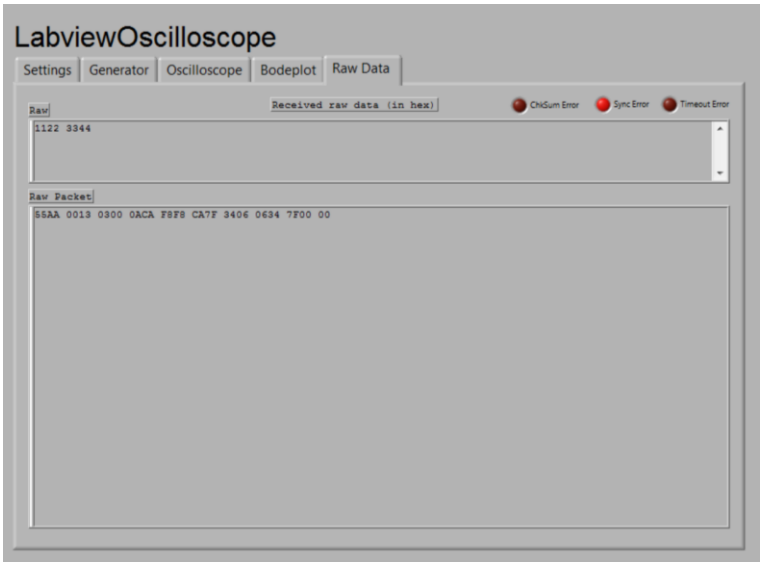


Figure 8 Under Raw Data kan man se de rå data fra microcontrolleren

Fanen viser to data felter Raw og Raw Packet. Data fra microcontrolleren kommer ind i disse to felter som hexadecimal data. Raw feltet viser data som de kommer ind fra microcontrolleren. I disse data leder LabviewOscilloscope efter datamønsteret 0x55AA (0x foran et tal betyder, at det er en hexadecimal værdi). Dette mønster indikerer starten af en datapakke (se afsnit 3.6). Når disse to byte findes læses resten af datapakken (ud fra længdefeltet i pakken). Den fulde pakke sættes herefter ind i Raw Packet feltet. Raw slettes og LabviewOscilloscope starter med at lede efter en ny datapakke. I *Figure 8* ses et eksempel hvor der er læst en fuld datapakke (afsluttende med 0x0000 som checksum) efterfulgt af fire databytes 0x11223344. De sidste fire er ikke en del af datapakkestrukturen (se Annex A) og vil blive ignoreret af programmet. Denne fejl er indikeret ved at ”Sync Error” lampen lyser. Lampen ”ChkSum Error” indikerer om der er en checksum-fejl i datapakken. Hvis man under opsætningen har valgt ”ZERO16” som checksum vil LabviewOscilloscope checkke at de to checksum bytes er lig med 0x0000 men ikke lave noget check af data i pakken. Lampen ”Timeout Error” indikerer hvis programmet har fundet datafeltet 0x55AA og forsøger at læse resten af pakken men at der ikke kommer en hel pakke indenfor den timeout der er sat under opsætningen.

Fanen ”Raw Data” er anvendelig under udvikling og fejlfinding af koden til microcontrolleren.

3.6 Data Protocol

Data sendes til og modtages fra LabviewOscilloscope via protokollen der er beskrevet i Annex A. Protokollen er baseret på en datapakke med strukturen: packet header, et datafelt og en packet footer.

Packet Header			**Packet Data**			**Packet Footer**
Sync	Length	Type	Data 1	…..	Data N	Checksum
2 bytes	2 bytes	1 byte	N bytes			2 bytes

`Packet Header` angiver starten af datapakken og består af 3 felter: `Sync`, `Length` og `Type`.
`Packet Data` er indholdet af datapakken og består af et antal brugerdefinerede databyte.
`Packet Footer` angiver enden af datapakken og består af feltet: `Checksum`.

En datapakke er defineret ved nedenstående:

- `Sync` har altid værdien 0x55AA og bruges til at indikere starten af en datapakke.
- `Length` angiver længden af hele pakken. Når `Sync` er fundet (0x55AA) ved man at de næste to byte angiver længden af pakken og man ved således hvor mange bytes man skal læse for at få hele pakken.
- `Checksum` passer med indholdet af datapakken.

Det er således ikke nødvendigt at se på indholdet (andet end ved checksum beregningen) for at kunne afgøre om det er en valid datapakke. Hvis de tre ovenstående ting er opfyldt i en strøm af bytes på seriel porten er man næsten sikker på at det er en valid datapakke. Sandsynligheden afhænger af algoritmen som checksum er baseret på.

Ovenstående protokolstruktur bruges både ved data der sendes til LabviewOscilloscope (også kaldet Telecommands) og ved data der sendes fra LabviewOscilloscope (også kaldet Telemetry).

Datafeltet `Type` beskriver indholdet af pakken.

	Type	Value	Data	Beskrivelse
Telemetry	BTN	0x01	2 bytes	Sendes fra LabviewOscilloscope når der er trykkes på en af BTN knapperne (afsnit 3.2)
	SEND	0x02	4 bytes	Sendes fra LabviewOscilloscope når der trykkes på SEND knappen (afsnit 3.3)
	START	0x03	0 bytes	Sendes fra LabviewOscilloscope når der trykkes på START knappen (afsnit 3.4)
Telecommands	GENERATOR	0x01	4 bytes	Sendes til LabviewOscilloscope når venstre side af generator fanen skal opdateres (afsnit 3.2)
	OSCILLOSCOPE	0x02	X bytes	Sendes til LabviewOscilloscope når oscilloskop grafen skal opdateres (afsnit 3.3)
	BODE PLOT	0x03	256 bytes	Sendes til LabviewOscilloscope når bode plot grafen skal opdateres (afsnit 3.4)

Indholdet af `Packet Data` feltet fremgår af protokolbeskrivelsen i Annex A.

4 Blokdiagram

Blokdiagrammet i et LabVIEW program indeholder beskrivelsen af programflow i form af et diagram i sproget G. Diagrammet består af forskellige blokke der udfører forskellige funktioner og som er forbundet ved hjælp af signalledninger som overfører information mellem blokkene. Blokdiagrammet bringes frem ved at trykke på Ctrl+E.

4.1 LabVIEW basics

I *Figure 9* ses frontpanel og blokdiagram for et simpelt LabVIEW program (også kaldet virtual instrument eller bare ”VI”).

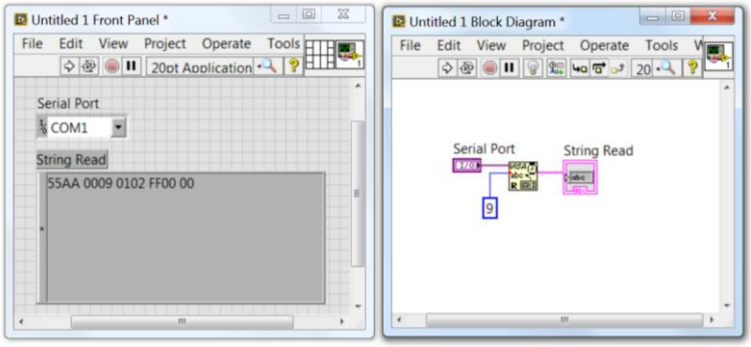


Figure 9 Simpel LabVIEW VI til at læse 9 bytes på seriel porten

Denne VI læser 9 bytes på serielporten angivet ved ”Serial Port” og lægger det ud i et datafelt kaldet ”String Read”. ”Serial Port” og ”String Read” er datafelter fra frontpanelet. ”Serial Port” er et ”Control” felt som brugeren kan ændre (input) mens ”String Read” er en ”Indicator” som VI’en selv opdaterer (output). I blokdiagrammet er boksen markeret med et R funktionen for at læse data fra serielporten. Man kan få et udvalg af funktioner at vælge mellem ved at højre clikke i blokdiagrammet.

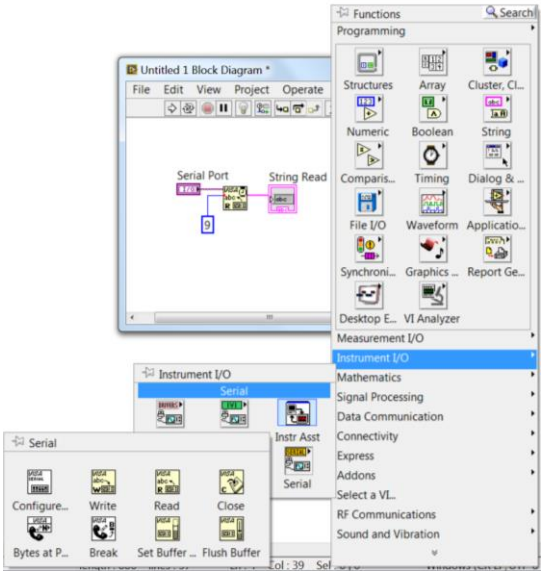


Figure 10 Funktionsoversigten i LabVIEW der giver et udvalg af funktioner

I *Figure 10* ses hvordan den serielle læsefunktion findes ved at vælge Instrument I/O >> Serial >> Read.

I *Figure 11* ses et andet eksempel. Her læses først en byte fra seriel porten. Denne byte tilføjes til ”Raw” indicator’en. ”Raw” er her brugt som en lokalvariabel (en kopi af indikatoren – originalen ses oppe i højre hjørne) først som en ”read” variabel som bliver ”concatenated” med byten fra seriel porten og bliver lagt ud i ”Raw” igen ved hjælp af en ”write” lokalvariabel. Byten fra serielporten bliver også sammenlignet med 0x55. Rækkefølgen af de to processer kan ikke på forhånd forudsiges og man skal forestille sig at de foregår samtidigt. De tre funktioner – læs serielporten, læg byten til Raw og sammenlign med 0x55 – er sat ind i en grå ramme. Denne ramme er en løkke der repeterer de tre funktioner. Hvis byten er lig med 0x55 aktiveres en rød ”stop” blok der får løkken til at stoppe og man går derefter videre. Det næste der sker er at der læses en ny byte og ”Raw bliver opdateret med de to seneste byte læst fra seriel porten. Hvis det er 0x55AA går man videre og lægger resultatet ind i control’en ”String”. Hvis ikke startes forfra med at lede efter 0x55.

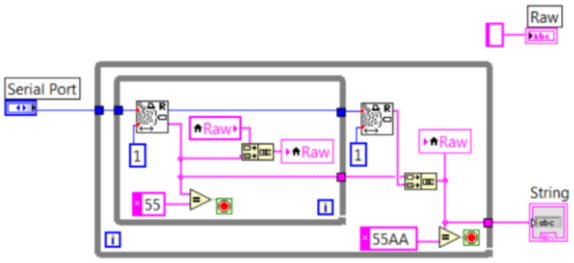


Figure 11 Eksempel på en VI der bruger løkker

I *Figure 12* ses et eksempel hvor ”Raw Packet” bliver læst ind i en betingelsesramme. Denne ramme indeholder et antal diagrammer der udføres afhængig af en betingelse. Man kan skifte mellem de betingede diagrammer ved at trykke på højre/venstre pilene i toppen af rammen (angivet med et 1-tal). Betingelsen ses i venstre side som et blåt spørgsmålstegn. Den er her forbundet til en ”typecast” og en ”string subset” boks. Ved at trykker på Ctrl+h bringes hjælpevinduet ”Context Help”. Hvis man føre curseren hen over ”string subset” boksen fås instruktionen som vist på *Figure 12*.

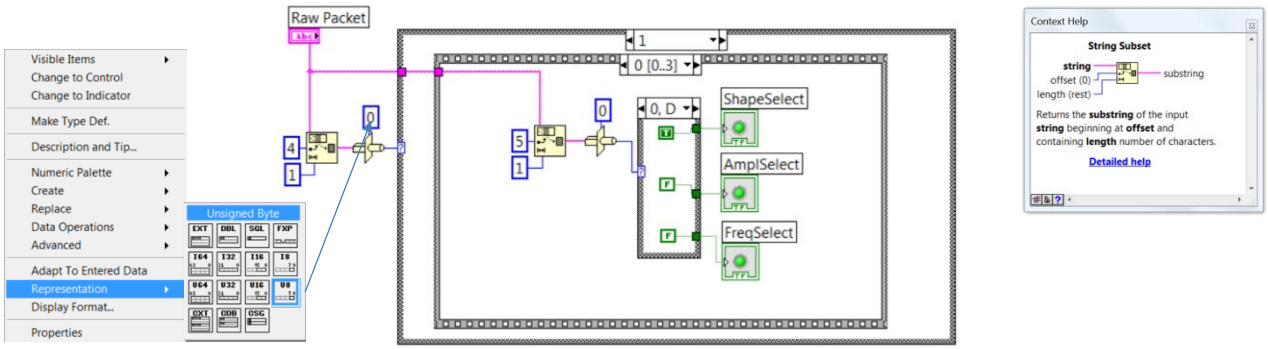


Figure 12 Eksempel på en VI der bruger betingelse og sequence

Der læses således en streng på plads 4 fra ”Raw Packet” og med en længde på 1, altså den 5. byte. Dette svare til ”Type” feltet (se afsnit 3.6) hvis ”Raw Packet” indeholder en datapakke sendt til LabviewOscilloscope. Denne byte type cast’es til et tal i byte-repræsentation. Tallet ”0” over type cast boksen bruges til at indikerer hvilken tal-repræsentation der konverteres til. Repræsentationen af et tal kan ses (og ændres) ved at højre-klikke på tallet og vælge ”Representation” (se *Figure 12*). Uformateret data håndteres ofte som strenge i Labview og type cast’es efterfølgende til et passende format i henhold til en given protokol. Man kan se hvordan ledningen gennem type-cast ændre sig fra lyserød til blå. Lyserøde ledninger overfører strenge, blå overfører integer, orange overfører real og grønne overfører boolean. Betingelsesrammen bliver således styret af Type-feltet

i en datapakke (Annex A). Hvis type-feltet er ”1” udføres det diagram som ses i *Figure 12*. De andre betingelsesrammer som udføres ved andre typer er skjult i *Figure 12*. Det første der udføres under ramme ”1” er en stacked sequence. Det er et antal rammer (i dette tilfælde fire) der udføres i sekvens. Den første ramme (sekvens 0) ses i *Figure 12*. Her oversættes byte nummer 5 og bruges i en anden betingelsesramme. Denne byte svare til ”Active indicator” i protokollen (Annex A). Denne byte bestemmer hvilken lampe i venstre side af Generator-fanen der skal lyse (afsnit 3.2). Hvis byten er 0 skal Shape-lampen lyse. I *Figure 12* ses at bool indicator’en ”ShapeSelect” får tildelt værdien T (true) mens ”AmplSelect” og ”FreqSelect” får tildelt værdien F (false). På frontpanelet er disse select-indicator’ere lamper der repræsenterer det aktive valg af parameter.

4.2 Telecommand Løkken

Blokdiagrammet for LabviewOscilloscope VI’en består af to repeat-løkker. Den øverste udgør Telecommand løkken og håndterer data der bliver sendt til VI’en via seriel porten (se *Figure 13*). Før løkken starter initialiseres porten. Indholdet af løkken går ikke i gang før alle input til løkken er genereret. Udgangen af ”VISA Serial” initialiseringen er ført til kanten af løkken og bevirker at løkken først starter når porten er initialiseret. Inde i løkken er diagrammet for ”Packet Header” syncronisation (som beskrevet i *Figure 11*) placeret. Når sync feltet (0x55AA) er fundet læses de to næste bytes som så angiver hvor mange bytes der efterfølgende skal læses (se afsnit 3.6). Den samlede datapakke bliver overført til ”Raw Packet” og bliver chekket for cheksum ved hjælp af sub-VI’en ChkSum.vi. Hvis checksum’en er verificeret udføres datapakkefortolkningen ud fra type-feltet i datapakken (svare til *Figure 12*).

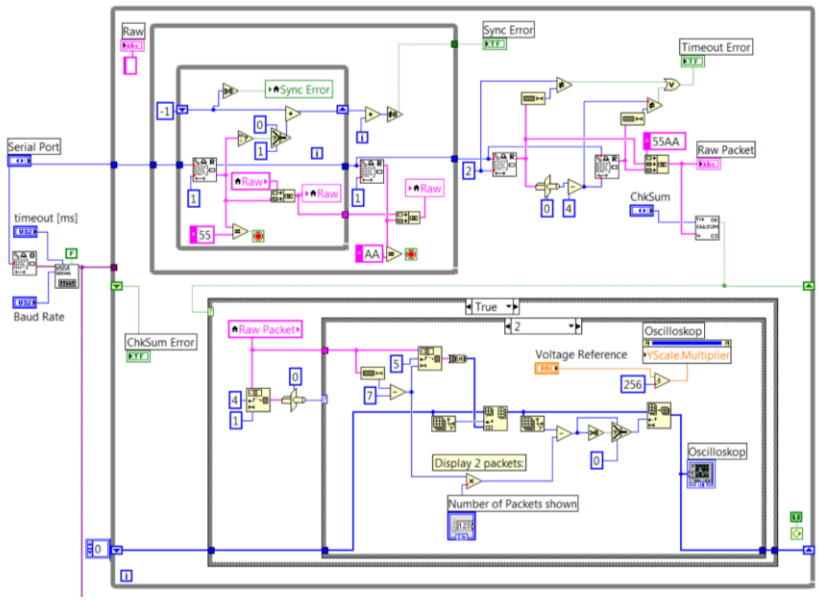


Figure 13 Telecommand løkken i LabviewOscilloscope

I *Figure 13* er fortolkningen af oscilloscop-data vist. Data til oscilloscop-plottet bliver sorteret ud af datapakken og bliver sat sammen med et data-array fra tidligere pakker. Data-array’et bliver beskåret i forhold til ”Number of Packets shown” før det bliver lagt ud i Oscilloscope indicator’en. Denne indicator repræsenterer oscilloscope plottet på frontpanelet (se afsnit 3.3). Data-arrayet bliver ført ud til kanten af den yderste repeat-løkke og forbundet med et shift-register. Næste gang løkken køre vil dette data-array blive ført ind i løkken på samme sted bare i venstre side. Nu repræsenterer det data-array fra tidligere pakker. Y-aksen på plottet bliver skaleret automatisk i forhold til værdien for spændingsreferencen som er angivet i setup-fanen (afsnit 3.1).

4.3 Telemetry Løkken

Nedenunder Telecommand løkken er løkken for Telemetry som håndterer data sendt fra Labview Oscilloscope (*Figure 14*). Løkken består af en stacked sequence der sørger for at sende en af de tre telemetry datapakker hvis der er trykket på en af knapperne. Den sequence der ses på *Figure 14* svare til en type 0x01 telemetry datapakke som sendes hvis der trykkes på en af de fire BTN knapper. Knaptrykket bliver omregnet fra fire booleans til en integer. Hvis dette er forskelligt fra 0 udføres den betingede ramme som er vist på *Figure 14*. Datapakken sættes sammen af Packet Header og de to data byte BTN og SW (se Annex A). Dernæst udregnes checksum og sendes afsted sammen med resten af datapakken til seriel porten. Til sidst nulstilles alle BTN-knapperne. I telemetry løkken er der en metronom-boks forbundet til en konstant på 100. Dette bevirker at løkken udføres hvert hundrede millisekund eller 10 gange i sekundet.

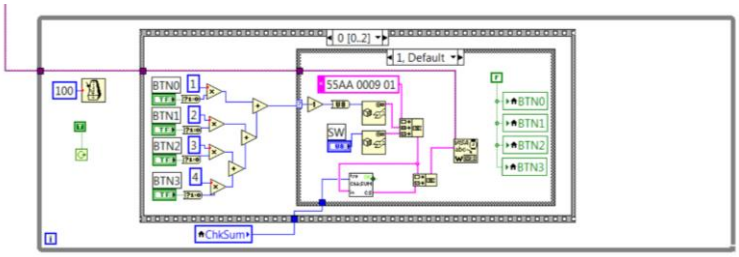


Figure 14 Telemetry løkken i LabviewOscilloscope

5 Checksum

Til check af checksum benyttes VI'en "LabviewOscilloscopeChecksum.vi". I *Figure 15* ses frontpanelet af denne VI. Datapakken indtastes i hexadecimal format og checksumtypen vælges. Der kan vælges mellem checksumtyperne: ZERO16, LRC8 eller CRC16-CCITT. Reslutatet kommer ud i feltet "ChkSum". Hvis datapakken indeholder de to checksum byte vil feltet "Checked" angive om det er en valid checksum eller om den er "failed".

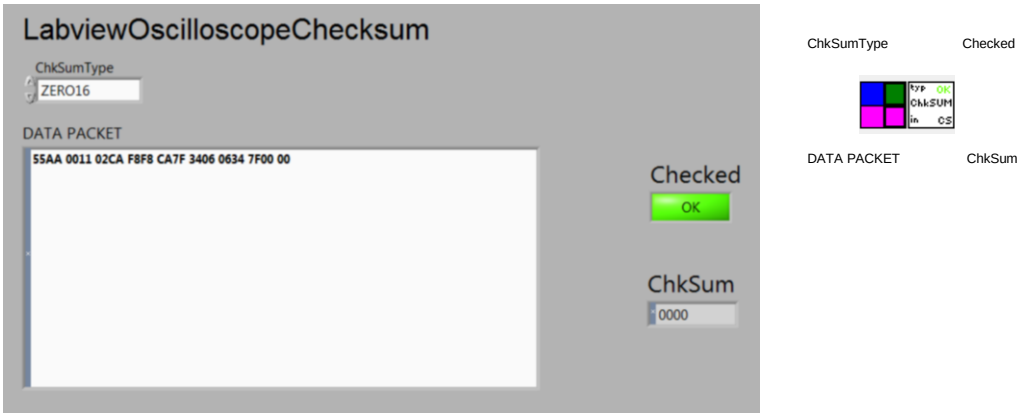


Figure 15 Frontpanel af LabviewOscilloscopeChecksum

Når denne VI bruges som sub-VI skal den forbindes som angivet øverst i venstre hjørne i *Figure 15*.

5.1 ZERO16

Blokdiagrammet for LabviewOscilloscopeChecksum består af en betingelsesramme baseret på ChkSum. I *Figure 16* ses diagrammet for algoritmen for ZERO16 checksum. Det undersøges om de to sidste bytes er lig med 0x0000 og denne værdi sættes ud som "ChkSum".

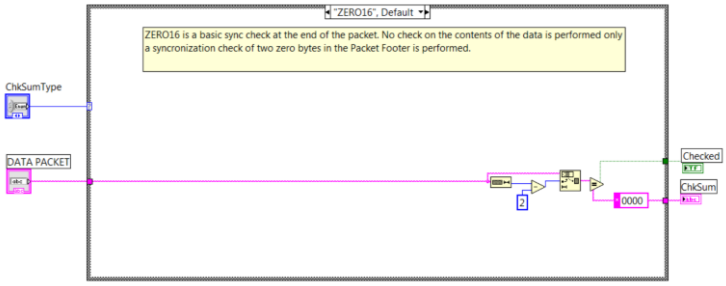


Figure 16 Blokdiagram for ZERO16 checksum

5.2 LRC8

I *Figure 17* ses diagrammet for algoritmen for LRC8 checksum. Dette er en "Longitudinal Redundancy Check" algoritme og er baseret på en sequentiel bitvis xor af alle byte i datapakken. Dette er den simpleste form for checksum der undersøger data i en pakke. Hvis der er en "bit-flip" i dataene vil denne algoritme kunne finde fejlen. Algoritmen har nogle svagheder. Den kan f.eks. ikke finde ekstra byte med værdien 0x00 og. Hvis der er en dobbelt bit-flip er sandsynligheden for at xor-algoritmen finder fejlen ca 90% mens en fire-double fejl kun findes i 50% af tilfældene.

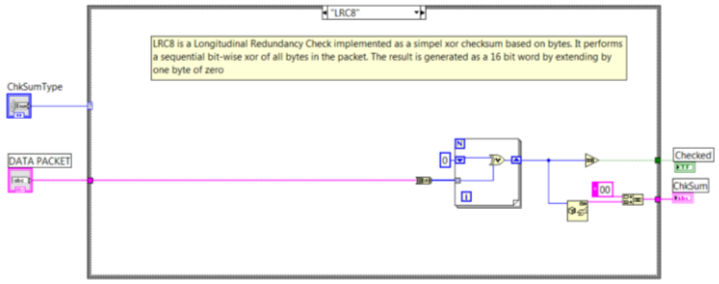


Figure 17 Blokdiagram for LRC8 checksum

Resultatet af algoritmen er en byte der lægges ud som den sidste af de to checksum byte. Den første byte sættes til værdien 0x00. Hvis en hel datapakke med en valid checksum køre igennem algoritmen skal resultatet være nul.

5.3 CRC16-CCITT

I *Figure 18* ses diagrammet for algoritmen for CRC16-CCITT checksum. Dette er en ”Cyclic Redundancy Check” algoritme og er baseret på en cyklisk division med et polynomium af alle bytes i datapakken. Denne algoritme er meget effektiv til at finde fejl i en datapakke og bruges mange steder (f.eks. bluetooth). Der findes mange varianter af denne algoritme.

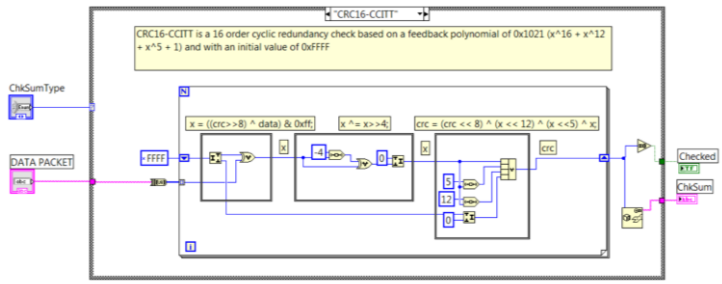


Figure 18 Blokdiagram for CRC16-CCITT checksum

Resultatet af algoritmen er et word (16-bit) der lægges ud som de to checksum byte. Hvis en hel datapakke med en valid checksum køre igennem algoritmen skal resultatet være nul.

6 Simulator

For at kunne teste protokollen til LabviewOscilloscope før den er implementeret i microcontrolleren er der lavet en simulator der simulerer programmet. Simulatoren ”SimulatorOscilloscope” er en kopi af den originale VI blot hvor seriel port funktionerne er skiftet ud med moduler der kommunikerer internt i computeren. Simulatoren står og venter på at der kommer en datapakke (ligesom originalen). For at kunne sende en Telecommand pakke er der også lavet en stimulator ”SimulatorControlOscilloscope”. Her kan man også se Telemetry pakker som sendes fra SimulatorOscilloscope når der trykkes på en knap. De to VI skal køre samtidig på samme computer.

6.1 SimulatorOscilloscope

I *Figure 19* ses frontpanel og blokdiagram af VI’en ”SimulatorOscilloscope”. Den er en kopi af LabviewOscilloscope men hvor seriel port funktionerne er skiftet ud. I blokdiagrammet ses røde funktioner hvor der normalt er læse eller skrive funktioner i originalen. Disse funktioner bruger nogle globale variable til at læse fra eller skrive til i stedet for seriel porten.

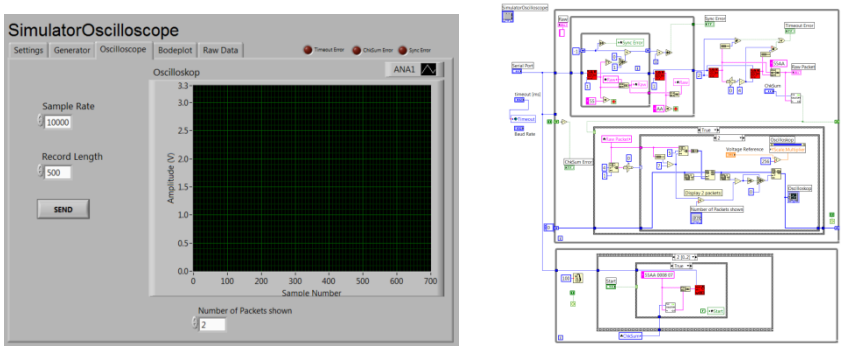


Figure 19 SimulatorOscilloscope komunikerer internt I computeren

6.2 SimulatorControlOscilloscope

I *Figure 20* ses frontpanel og blokdiagram af VI’en ”SimulatorControlOscilloscope”. Feltet ”Telecommand Packet” bruges til at skrive en datapakke til SimulatorOscilloscope.



Figure 20 SimulatorControlOscilloscope kan sende og modtage datapakker fra simulatoren

Det kunne f.eks. være en oscilloskoppakke som nedenstående:

55AA001102CAF8F8CA7F340606347F0000

Denne pakke begynder med sync feltet (0x55AA). Den indeholder 17 bytes (0x0011) og er en type 2 (0x02) pakke som svare til en oscilloskoppakke. Der er 10 databyte og den ender med to checksum byte på nul (0x0000). Denne pakke kan kopieres ind i Telecommand Packet feltet. Når der trykkes ”SEND” vil pakken overføres til den globale variabel som SimulatorOscilloscope læser fra og den vil fortolke pakken som en oscilloskoppakke.

Hvis der trykkes på en af knapperne i SimulatorOscilloscope vil der sendes en datapakke som kan ses i feltet ”Telemetry from LabviewOscilloscope”.

I *Figure 21* ses hierarkiet af VI’s som skal bruges i forbindelse med simulatoren. Filerne for disse VI’s skal være placeret i samme folder når man stater de to simulator VI’s.

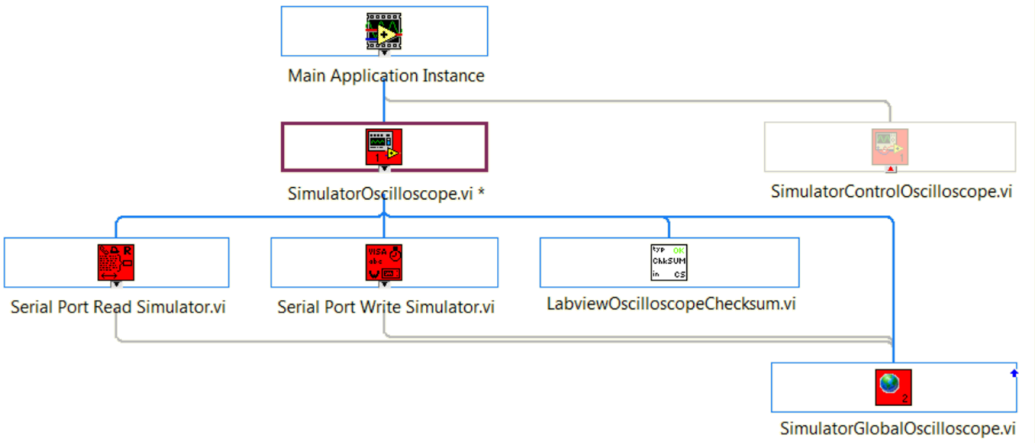


Figure 21 VI Hierarchy for Simulatoren

Annex A Protocol description

The overall protocol for the communication between LabviewOscilloscope and the microcontroller (both directions) is defined below:

Size	Content
2 bytes	Sync (0x55AA)
2 bytes	Length of entire packet
1 byte	Type (see below)
x bytes	Data
2 bytes	CRC (or zeros)

Big endian is used for transmission of multi byte words. E.g. to transfer a value of 0x1020, the byte 0x10 is transferred first followed by 0x20.
Three different checksum algorithms is implemented (XOR8, CRC16-CCITT or CRC16-KERMIT). In XOR8 only the last byte of the CRC field is used.

A.1 Telemetry from LabviewOscilloscope

The Type field in telemetry packets can have the following values:

Type Value	Meaning
0x01	BTN button pressed in Generator tab
0x02	SEND pressed in Oscilloscope tab
0x03	START pressed in Bode Plot tab

A.1.1 Type 1 Telemetry

The Data field will contain the following for a BTN packet:

Size	Content
1 byte	BTN Value*
1 byte	SW Value

* *0x00~BTN0, 0x01~BTN1, 0x02~BTN2, 0x03~BTN3*

A.1.2 Type 2 Telemetry

The Data field will contain the following for a SEND packet:

Size	Content
2 bytes	Sample rate
2 bytes	Record length

The sample rate is given in samples pr second.

The record length is the number of points on the oscilloscope graph and may be used as the number of samples in the “oscilloscope data” packet as described below

A.1.3 Type 3 Telemetry

The Data field is empty for this telemetry packet.

Examples (in hexadecimal):

0x55 0xAA 0x00 0x09 0x01 0x02 0xFF 0x00 0x00
BTN2 pressed with a SW value of 20h (no checksum)

0x55 0xAA 0x00 0x0B 0x02 0x27 0x10 0x01 0xF4 0x00 0x34
SEND pressed with 10000 sample rate and 500 record length (LRC8 checksum)

A.2 Telecommands to LabviewOscilloscope

The Type field in telecommand packets can have the following values:

Type Value	Meaning
0x01	GENERATOR data
0x02	OSCILLOSCOPE data
0x03	BODE PLOT data

A.2.1 Type 1 Telecommands

The Data field will contain the following bytes for a GENERATOR packet:

Size	Content
1 byte	Active indicator
1 byte	Shape
1 byte	Amplitude
1 byte	Frequency

This packet will update the left side of the Generator tab in the front panel of LabviewOcilloscope.

A.2.2 Type 2 Telecommands

The Data field will contain the following bytes for an OSCILLOSCOPE packet:

Size	Content
x bytes	Sample data (one byte per sample)

A.2.3 Type 3 Telecommands

The Data field will contain the following bytes for a BODE PLOT packet:

Size	Content
255 bytes	Amplitude data (one byte per sample)

Amplitude data is relative to the first amplitude byte. The values are given in the range of 0x00 – 0xFF with 0xFF corresponding to a relative value of 1 and 0x00 corresponding to a relative value of 1/256.

Examples (in hexadecimal):

0x55 0xAA 0x00 0x0B 0x01 0x01 0x02 0x41 0x65 0x00 0x00
Active indicator = 1, shape = 2, amplitude = 65 (41h), frequency = 101 (65h) – no checksum

0x55 0xAA 0x00 0x11 0x02 0xCA 0xF8 0xF8 0xCA 0x7F 0x34 0x06 0x06 0x34 0x7F 0x63 0x92
Oscilloscope data, 10 samples – CRC16 checksum

0x55AA016502818486888A8D8F919496989A9C9FA1A3A5A7AAACAEB0B2B4B6B8BABCBEC0
C2C4C6C8CACCCECFD1D3D5D6D8DADBDDDEE0E1E3E4E5E7E8E9EBECEDEEEFF0F1F2F
3F4F5F6F7F8F8F9FAFAFBFBFCFCFDFDFDFEFEFEFEFEFEFEFEFEFEFEFEFEFEFEFDFDFDFC
FCFBFBFAFAF9F8F8F7F6F5F4F3F2F1F0EFEEEDECEBE9E8E7E5E4E3E1E0DEDDDBDAD8D
6D5D3D1CFCECCCAC8C6C4C2C0BEBCBAB8B6B4B2B0AEACAAA7A5A3A19F9C9A98969491
8F8D8A888684817F7D7A787674716F6D6A686664625F5D5B59575452504E4C4A48464442403
E3C3A38363432302F2D2B292826242321201E1D1B1A19171615131211100F0E0D0C0B0A0908
0706060504040303020201010100000000000000000000000000010101020203030404050606
0708090A0B0C0D0E0F10111213151617191A1B1D1E202123242628292B2D2F30323436383A3
C3E40424446484A4C4E50525457595B5D5F626466686A6D6F717476787A7D7F0000

Oscilloscope data, 350 samples – no checksum